

Scala

scala is similar to python

val and var are the two variable types in scala

Scala allows one to decide whether a variable is immutable or mutable

Immutable; -"val" (read only)

Similar to java final variable

```
Val msg="hello world"
```

```
out=string ="hello world"
```

```
scala> msg="Hello"
```

```
^
```

error: reassignment to val

Mutable - 'var' (read-write)

Similar to non final variables in java

```
scala> var msg="Hello world"
```

```
var msg: String = Hello world
```

```
scala> msg="Hello"
```

```
// mutated msg
```

Even though we never declared variable type, scala inferred it

This is called as **Type Inference** in Scala

```
scala> var msg="Hello world"
```

```
var msg: String = Hello world
```

```
scala> msg=15
```

```
^
```

error: type mismatch;

found : Int(15)

required: String

Once a type is assigned to a variable, it remains same for entire scope

In scala, a **{ }** **block** is a list of expressions and result is also an expression

The value of a block is the value of the last expression of it.

```
scala> val x={val a=10;val b=100;b-a}
```

```
val x: Int = 90
```

```
scala> val x={val a=10;val b=100;val c=20;(b-a)*c}
```

```
val x: Int = 1800
```

If -else syntax in scala is same as java or c++
Semicolons are optional in scala

```
scala> var x=5
var x: Int = 5
scala> val s=if (x>0 && x<6)1 else 0
val s: Int = 1
```

```
scala> val s=if (x>0 && x<6)"positive" else "negative"
val s: String = positive
```

```
scala> val s=if (x>0 && x<6)"positive" else 0
val s: Any = positive
```

Every expression in scala has a type
First if statement has a type Int other one has Any.type mixed expression is supertype of both branches

While and do while loops are same java

```
var args="500"
var args: String = 500
scala> var i=0
var i: Int = 0
scala> while(i<args.length)
  | {
  | println(args(i))
  | i+=1
  | }
5
0
0
l++ will not work here use i+=1
```

```
scala> var i=0
var i: Int = 0
scala> do
  | {println(args(i))
  | i+=1
  | }while(i<(args.length))
5
0
0
```

For each loop

```
scala> var args ="Hello"
var args: String = Hello
```

```
scala> args.foreach(args => println(args))
```

```
H
```

```
E
```

```
I
```

```
I
```

```
O
```

```
scala> args.foreach(println)
```

```
H
```

```
E
```

```
I
```

```
I
```

```
O
```

For loop

Scala doesn't have for(initialise;test;update)syntax

Either you'll use while loop or a statement like below

```
scala> for (i<-1 to 5)
```

```
  | println(i)
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

While traversing an array, following could be applied:

```
scala> val in ="hello world"
```

```
val in: String = hello world
```

```
scala> var sum =0
```

```
var sum: Int = 0
```

```
scala> for (i<-0 until in.length)
```

```
  | sum+=i
```

```
scala> print(sum)
```

```
55
```

Print this

```
11
```

```
12
```

```
13
```

```
21
```

```
22
```

```
23
```

```
31
```

```
32
```

```
33
```

```
for (i<-1 to 3)
```

```
  | for (j<-1 to 3)println(i*10+j)
```

Multiple generator

```
for (i<-1 to 3;j<-1 to 3) println(10*i+j)
11
12
13
21
22
23
31
32
33
for (i<-1 to 3;x=4-i;j<-x to 3) println(10*i+j)
13
22
23
31
32
33
for (i<-1 to 3;j<-1 to 3 if i==j) println(10*i+j)
11
22
33
```

Palindrome

```
while(i<args.length){
  | println(args((args.length)-i-1))
  | i=i+1}
o
l
l
e
h
```

Function

Area of circle

```
def area (radius:Double):Double ={3.14*radius*radius}
def area(radius: Double): Double
```

```
scala> area(10.9)
val res4: Double = 373.0634
```

factorial

```
def fact(n:Int):Int = if(n==0)1 else n*fact(n-1)
def fact(n: Int): Int
```

```
scala> fact(5)
```

```
val res6: Int = 120
```

Assignments

Prime

```
def prime (n:Int):Boolean ={  
  | if (n<=1) false  
  | else if (n==2) true  
  | else !(2 to math.sqrt(n).toInt).exists(n%_==0)}  
def prime(n: Int): Boolean
```

```
scala> prime(9)  
val res23: Boolean = false
```

```
scala> prime(8)  
val res24: Boolean = false
```

```
scala> prime(7)  
val res25: Boolean = true
```

Area of rectangle

```
def area_r(length:Double,breadth:Double):Double={length*breadth}  
def area_r(length: Double, breadth: Double): Double
```

```
scala> area_r(10,12)  
val res5: Double = 120.0
```

Odd or even

```
def odd(n:Int):String= if(n%2!=0)"odd" else "even"  
def odd(n: Int): String
```

```
scala> odd(3)  
val res13: String = odd
```

```
scala> odd(14)  
val res14: String = even
```

Concat

```
def concatstr(arg1:String,arg2:String = "vishal",arg3:String= "manhor")=print(arg1+"  
"+arg2+" "+arg3)  
def concatstr(arg1: String, arg2: String, arg3: String): Unit
```

```
scala> concatstr("hi")  
hi vishal manhor
```

```
def rect_area(length:Float,breadth:Float):Unit = {val area =length*breadth;print(area  
)}  
def rect_area(length: Float, breadth: Float): Unit
```

```
scala> rect_area(10,2)
20.0
```

Collections

Array

Arraybuffer

Map

Fixed length Arrays

```
val n= new Array[Int](10)
val n: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

```
scala> val s= new Array[String](10)
val s: Array[String] = Array(null, null, null, null, null, null, null, null, null, null)
```

```
scala> val st = Array("Hello","world")
val st: Array[String] = Array(Hello, world)
val s= new Array(10)
val s: Array[Nothing] = Array(null, null, null, null, null, null, null, null, null, null)
```

Scala collections ArrayBuffer

```
scala> import scala.collection.mutable.ArrayBuffer
import scala.collection.mutable.ArrayBuffer
```

```
scala> val a=ArrayBuffer[Int]()
val a: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer()
```

```
scala> a+=1
val res27: a.type = ArrayBuffer(1)
```

```
scala> a
val res28: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer(1)
```

```
scala> a+=(2,3,5)
^
```

warning: method += in trait Growable is deprecated (since 2.13.0): Use `++=` aka `addAll` instead of varargs `+=`; infix operations with an operand of multiple args will be deprecated

```
val res29: a.type = ArrayBuffer(1, 2, 3, 5)
```

```
scala> a++=Array(6,7,8)
val res30: a.type = ArrayBuffer(1, 2, 3, 5, 6, 7, 8)
```

a.trimEnd(2)

^

warning: method trimEnd in trait Buffer is deprecated (since 2.13.4): use dropRightInPlace instead

```
scala> print(a)
ArrayBuffer(1, 2, 3, 5, 6)
#2 elements are deleted
```

```
a.insert(3,10) #elements 3,10 are added
a.remove(2,3)
```

```
scala> print(a)
ArrayBuffer(1, 2)
#from 2nd index remove 3 elements
```

```
Array(1,2,3,4).sum
val res16: Int = 10
```

```
scala> Array(1,5,9,8).max
val res17: Int = 9
```

```
val a=Array(1,7,2,9)
val a: Array[Int] = Array(1, 7, 2, 9)
```

```
scala> scala.util.Sorting.quickSort(a)
```

```
scala> a.mkString(" ** ")
val res20: String = 1 ** 2 ** 7 ** 9
```

Map

In scala map is a collection of pair
A pair is a group of two values

```
scala> val mapping =Map("Vishal"->"kumar","vijay"->"verma")
val mapping: scala.collection.immutable.Map[String,String] = Map(Vishal -> kumar, vijay -> verma)
```

```
scala> val mapping=scala.collection.mutable.Map("Vishal"->"K","vijay"->"V")
val mapping: scala.collection.mutable.Map[String,String] = HashMap(Vishal -> K, vijay -> V)
```

Accessing Maps:

```
scala> val x=mapping("Vishal")
val x: String = K
```

```
scala> val x=mapping.getOrElse("vish",0)
val x: Any = 0
```

```
scala> mapping -= "vishal"
val res21: mapping.type = HashMap(Vishal -> K, vijay -> V)
```

```
scala> mapping -= "Vishal"
val res22: mapping.type = HashMap(vijay -> V)
```

```
scala> mapping += ("Ajay" -> "Sharma")
val res23: mapping.type = HashMap(Ajay -> Sharma, vijay -> V)
```

Tuple

In tuple numbering starts from 1 not 0

```
val a = (1, 4, "Bob", "Jack")
val a: (Int, Int, String, String) = (1, 4, Bob, Jack)
```

```
scala> a._2
val res24: Int = 4
```

```
"New Delhi India".partition(_._isUpper)
val res26: (String, String) = (NDI, ew elhi ndia)
```

List

List is either Nil or a combination of head and tail elements where tail is again a List

```
val lst = List(1, 2)
val lst: List[Int] = List(1, 2)
```

```
scala> lst.head
val res28: Int = 1
```

```
scala> lst.tail
val res29: List[Int] = List(2)
```

```
scala> 2 :: List(4, 5)
val res30: List[Int] = List(2, 4, 5)
```

We can use iterate over a list, but recursion is preferred practice in scala

```
def sum(l: List[Int]): Int = {if(l == Nil) 0 else l.head + sum(l.tail)}
def sum(l: List[Int]): Int
scala> val y = sum(lst)
val y: Int = 3
```

```
def p(l: List[Int]): Int = {if(l == Nil) 0 else l.head * sum(l.tail)}
def p(l: List[Int]): Int
scala> p(lst)
val res31: Int = 2
```


#assignments

- Write a scala program which produces all values from an array after removing duplicates
- Create a Map of prices for a number of shopping items you want. Now create another map from it with same key ,but prices slashed by 10%
- write a function minmax(values:Array[Int] which returns a pair of minimum and maximum values of an array passed to it.

Repeat the above exercise as procedure which prints the min max values of an array